Base Station GUI2 Documentation

How to use the Base Station GUI: Seth Ricks, July 2025

Welcome to the Base Station GUI Documentation. This documentation is designed to be a quick start for using and understanding the Base Station GUI. The following sub pages are included:

👀 How it looks >

This is a visual for how the GUI currently looks.

🐕‍🦺 GUI Layout Guide >

A guide for locating aspects of the GUI in the code for editing.

🧠 GUI Signals Mind Map >

A useful Mind Map for navigating all the topics and signals that the GUI utilizes.

GUI Code >

A rundown of the innards of the GUI, including starting it up and all of the files and folders that are used.

GUI Features >

Another rundown of the various features that the GUI has and how to use them.

🔨 GUI TODO >

What needs to be done on the GUI. Processes that started but have not been finished.

NOTE: Most of these pages have sub-pages. Make sure to check these out as well!

If you are interested in editing the GUI, it would be helpful to know how PyQt works. You can learn about that here: https://www.pythonguis.com/tutorials/pyqt6-first-steps-qt-designer/ ↗

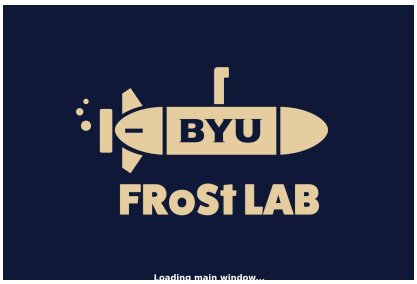
You will also need to understand how ROS works and have it install on your computer. You can learn about that here: https://docs.ros.org/en/humble/index.html ↗

If you have any questions, you can reach me at: sethricks340@gmail.com

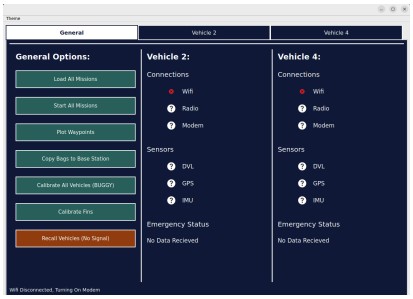
How it looks

Seth Ricks

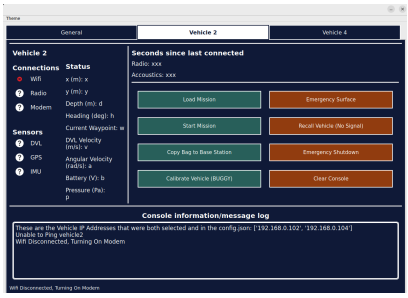
Updated 7/18/2025



Splash Screen (Dark Mode Default)



General Page



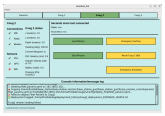
Specific Vehicle Page

Note: Tabs are based off of what vehicles you chose in the configuration window.

Themes: (layouts not updated, but colors are the same)

Dark Mode

Light Mode



Blue Pastel

Brown Sepia

Intense Dark

Intense Light

Cadetblue

GUI Layout Guide

Seth Ricks

The GUI that I have written with PyQt uses layouts and widgets extensively, which is represented by the red boxes in the images below. If you want to see these borders yourself, go to the "main" function in GUI_ros_node.py and change this line:

```
app, result, selected_cougs =
base_station_gui2.tabbed_window.OpenWindow(None, borders=False)
```

To this:

```
app, result, selected_cougs =
base_station_gui2.tabbed_window.OpenWindow(None, borders=True)
```

By changing the 'borders' parameter from False to True.

See the following sub pages for the Guides of the Layouts:

- 💂 GUI General Page Layout >
- 👓 GUI Specific Page Layout >
- 💦 Splash Screen Layout >

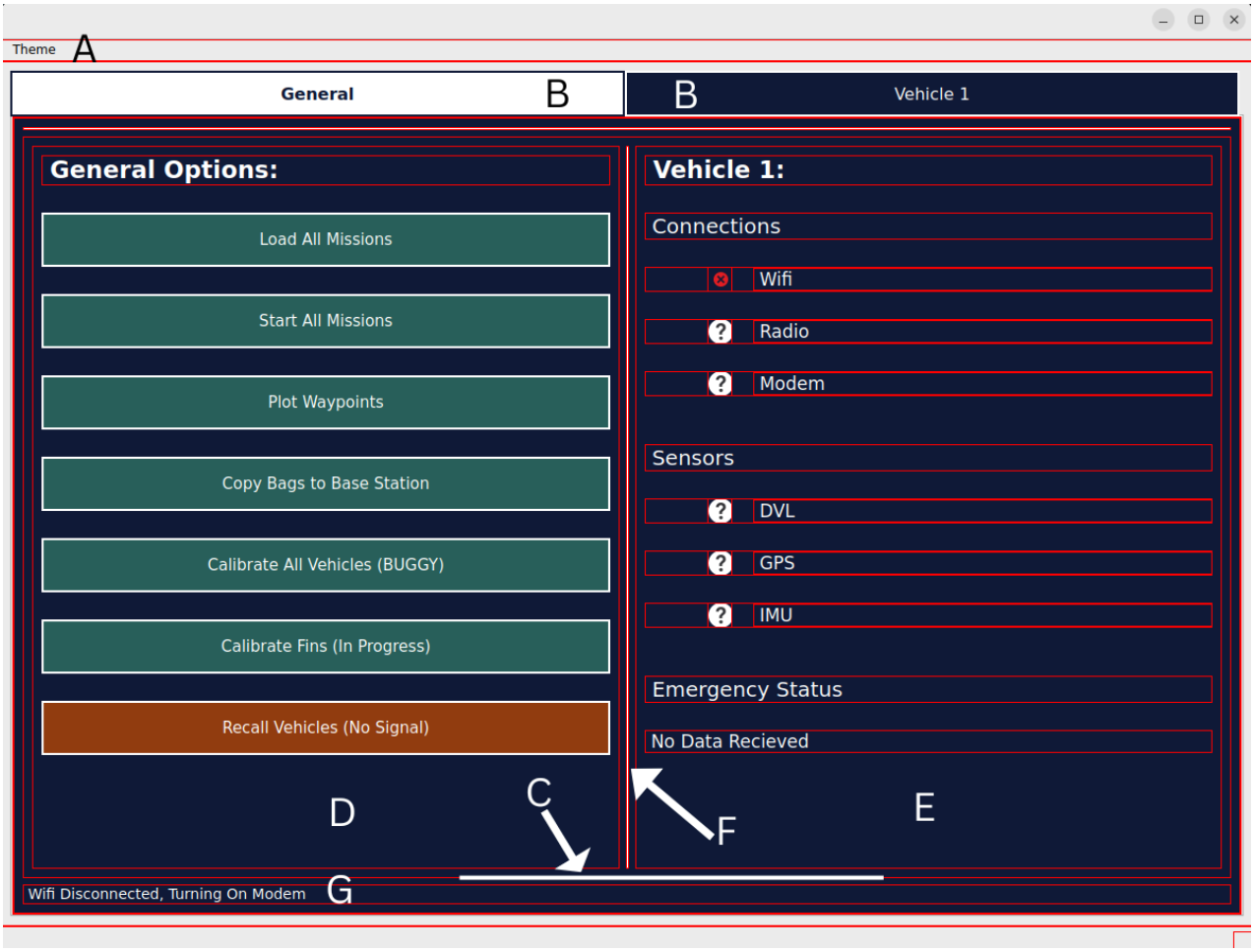
Note: If you are looking to change the logic of how the features of the GUI work, rather than changing the layout, there is a separate documentation for that. You can find that here:

- 🧠 GUI Features Mind Map >

GUI General Page Layout

Seth Ricks

The purpose of this page is to point you in the direction of where most of the layouts are created in the code. For example, if you wanted to add a label under the "Modem" widget, you would look at the E column in the image below and the description of where to find its creation in tabbed_window.py.



A:

```
class MainWindow(QMainWindow):
...
    def __init__(self, ros_node, vehicle_list):
    ...
    # This is where 'A' is created
    # Look around it to see where it is configured
    menu = self.menuBar()
    ...
```

B:

```
class MainWindow(QMainWindow):
...
    def __init__(self, ros_node, vehicle_list):
    ...

    # From...
    self.tabs = QTabWidget()
    ...
    # Up to the line ...
    ...

self.tabs.currentChanged.connect(self.scroll_console_to_bottom_on_tab)
```

C: (The box containing D, E, and F)

C is created in set_general_page_widgets, which also created D and E

```
class MainWindow(QMainWindow):
...
    def set_general_page_widgets(self):
        # Whole method is relevant
```

D: (Left column, with all the buttons)

```
class MainWindow(QMainWindow):
...
    def set_general_page_C0_widgets(self):
        # Whole method is relevant
```

E: (Right column, with all of the labels)

```
class MainWindow(QMainWindow):
...
    def set_general_page_C1_widgets(self):
        # Whole method is relevant
```

F: (The vertical line separating D and E)

A "vline" is created inbetween the general options and each vehicle in the method set_general_page_widgets

```
class MainWindow(QMainWindow):
...
    def set_general_page_widgets(self):
        ...
        # For each selected Vehicle, create a column and add to the
layout, separated by vertical lines
        for idx, vehicle_number in enumerate(self.selected_vehicles):
            self.general_page_layout.addWidget(self.make_vline())
            ...
        ...
```

G: (The confirm/reject label at the bottom)

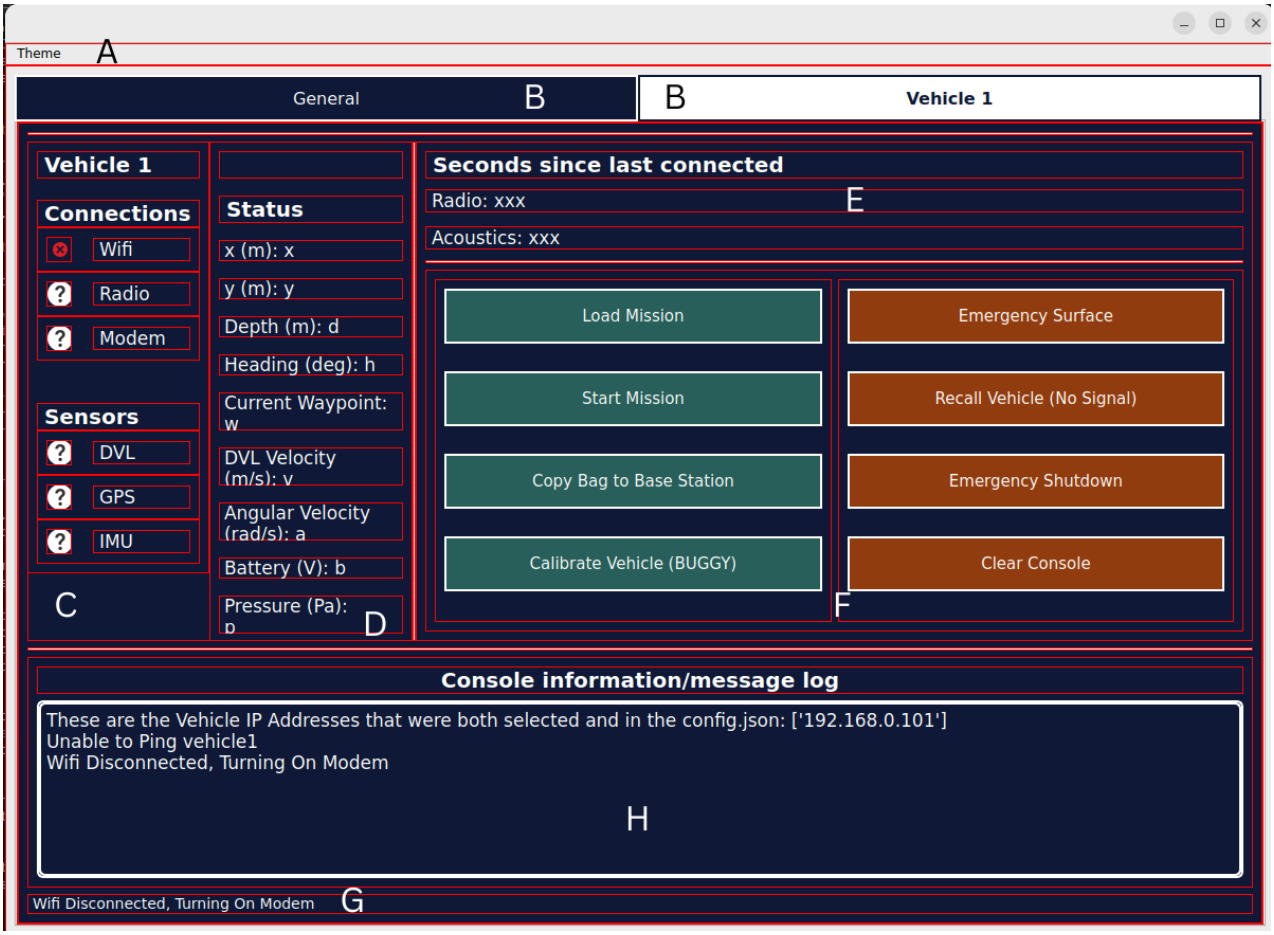
A separate label is created for each page in the init method.

```
class MainWindow(QMainWindow):
...
    def __init__(self, ros_node, vehicle_list):
        ...
        for name in self.tab_dict:
            ...
            # Add horizontal line and confirmation/rejection label
            content_layout.addWidget(self.make_hline())
            label = QLabel("Confirmation/Rejection messages from
command buttons will appear here")
            label.setStyleSheet(f"color: {self.text_color}; font-size:
14px;")
            self.confirm_reject_labels[name] = label
            ...
```

GUI Specific Page Layout

Seth Ricks

The purpose of this page is to point you in the direction of where most of the layouts are created in the code. For example, if you wanted to add a button under the "danger" buttons (the red/orange ones) then you would look at section F on this page. You would then go to the associated lines in tabbed_window.py.



A: Same as General Page Layout

B: Same as General Page Layout

C:

```
def create_specific_vehicle_column0(self, vehicle_number):
    ...
    # Add connection status icons (Wifi, Radio, Modem)
    wifi_widget = self.create_icon_and_text("Wifi",
self.icons_dict[self.feedback_dict["Wifi"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(wifi_widget)

    radio_widget = self.create_icon_and_text("Radio",
self.icons_dict[self.feedback_dict["Radio"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(radio_widget)

    modem_widget = self.create_icon_and_text("Modem",
self.icons_dict[self.feedback_dict["Modem"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(modem_widget)
    temp_layout.addSpacing(20)

    # Section: Sensors
    temp_label = QLabel("Sensors")
    temp_label.setFont(QFont("Arial", 15, QFont.Weight.Bold))
    temp_label.setStyleSheet(f"color: {self.text_color};")
    temp_label.setSizePolicy(QSizePolicy.Policy.Preferred,
QSizePolicy.Policy.Fixed)
    temp_layout.addSpacing(20)
    temp_layout.addWidget(temp_label)

    # Add sensor status icons (DVL, GPS, IMU)
    DVL_sensor_widget = self.create_icon_and_text("DVL",
self.icons_dict[self.feedback_dict["DVL"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(DVL_sensor_widget)

    GPS_sensor_widget = self.create_icon_and_text("GPS",
self.icons_dict[self.feedback_dict["GPS"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(GPS_sensor_widget)

    IMU_sensor_widget = self.create_icon_and_text("IMU",
self.icons_dict[self.feedback_dict["IMU"][vehicle_number]], 0,
vehicle_number, 1)
    temp_layout.addWidget(IMU_sensor_widget)
    ...
```

D:

```
def create_specific_vehicle_column01(self, vehicle_number):
    ...
    # Status widgets section
    status_spacing = 10
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_title_label(f"Status"),
alignment=Qt.AlignmentFlag.AlignTop)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("x (m): x",
f"XPos{vehicle_number}"), alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("y (m): y",
f"YPos{vehicle_number}"), alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Depth (m): d",
f"Depth{vehicle_number}"), alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Heading (deg): h",
f"Heading{vehicle_number}"), alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Current Waypoint:
w", f"Waypoint{vehicle_number}"),
alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("DVL Velocity <br>
(m/s): v", f"DVL_vel{vehicle_number}"),
alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Angular Velocity
<br>(rad/s): a", f"Angular_vel{vehicle_number}"),
alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Battery (V): b",
f"Battery{vehicle_number}"), alignment=Qt.AlignmentFlag.AlignVCenter)
    temp_layout.addSpacing(status_spacing)
    temp_layout.addWidget(self.create_normal_label("Pressure (Pa):
<br>p", f"Pressure{vehicle_number}"),
alignment=Qt.AlignmentFlag.AlignVCenter)
    ...
```

E:

```
def create_vehicle_buttons_column(self, vehicle_number):
    ...
    # Add a title label and connection time labels to the vertical
layout
    temp_V_layout.addWidget(self.create_title_label("Seconds since last
connected"))
    self.insert_label(temp_V_layout, "Radio: xxx", vehicle_number, 1)
    self.insert_label(temp_V_layout, "Acoustics: xxx", vehicle_number,
0)
    temp_V_layout.addWidget(self.make_hline())
    ...
```

F:

```
def create_vehicle_buttons_column(self, vehicle_number):
    ...
    #load mission (normal button)
    self.create_vehicle_button(vehicle_number, "load_mission", "Load
Mission", lambda: self.spec_load_missions_button(vehicle_number))
    #start mission (normal button)
    self.create_vehicle_button(vehicle_number, "start_mission", "Start
Mission", lambda: self.spec_start_missions_button(vehicle_number))
    #start mission (normal button)
    self.create_vehicle_button(vehicle_number, "copy_bag", "Copy Bag to
Base Station", lambda: self.spec_copy_bags(vehicle_number))
    #sync vehicle (normal button)
    self.create_vehicle_button(vehicle_number, "sync", "Calibrate
Vehicle (BUGGY)", lambda: self.run_calibrate_script(vehicle_number))

    #emergency surface (danger button)
    self.create_vehicle_button(vehicle_number, "emergency_surface",
"Emergency Surface", lambda:
self.emergency_surface_button(vehicle_number), danger=True)
    #abort mission (danger button)
    self.create_vehicle_button(vehicle_number, "recall", f"Recall
Vehicle (No Signal)", lambda: self.recall_spec_vehicle(vehicle_number),
danger=True)
    #emergency shutdown (danger button)
    self.create_vehicle_button(vehicle_number, "emergency_shutdown",
"Emergency Shutdown", lambda:
self.emergency_shutdown_button(vehicle_number), danger=True)
    #clear console (danger button)
    self.create_vehicle_button(vehicle_number, "clear_console", "Clear
Console", lambda: self.clear_console(vehicle_number), danger=True)

    temp_spacing = 20
    # Add buttons to the first and second sub-columns with spacing
    temp_layout1.addWidget(getattr(self,
f"load_mission_vehicle{vehicle_number}_button"))
    temp_layout1.addSpacing(temp_spacing)
    temp_layout1.addWidget(getattr(self,
f"start_mission_vehicle{vehicle_number}_button"))
    temp_layout1.addSpacing(temp_spacing)
    temp_layout1.addWidget(getattr(self,
f"copy_bag_vehicle{vehicle_number}_button"))
    temp_layout1.addSpacing(temp_spacing)
    temp_layout1.addWidget(getattr(self,
f"sync_vehicle{vehicle_number}_button"))
    temp_layout1.addSpacing(temp_spacing)
    temp_layout2.addWidget(getattr(self,
f"emergency_surface_vehicle{vehicle_number}_button"))
    temp_layout2.addSpacing(temp_spacing)
```

```
    temp_layout2.addWidget(getattr(self,
f"recall_vehicle{vehicle_number}_button"))
    temp_layout2.addSpacing(temp_spacing)
    temp_layout2.addWidget(getattr(self,
f"emergency_shutdown_vehicle{vehicle_number}_button"))
    temp_layout2.addSpacing(temp_spacing)
    temp_layout2.addWidget(getattr(self,
f"clear_console_vehicle{vehicle_number}_button"))
    ...
```

G: Same as General Page Layout

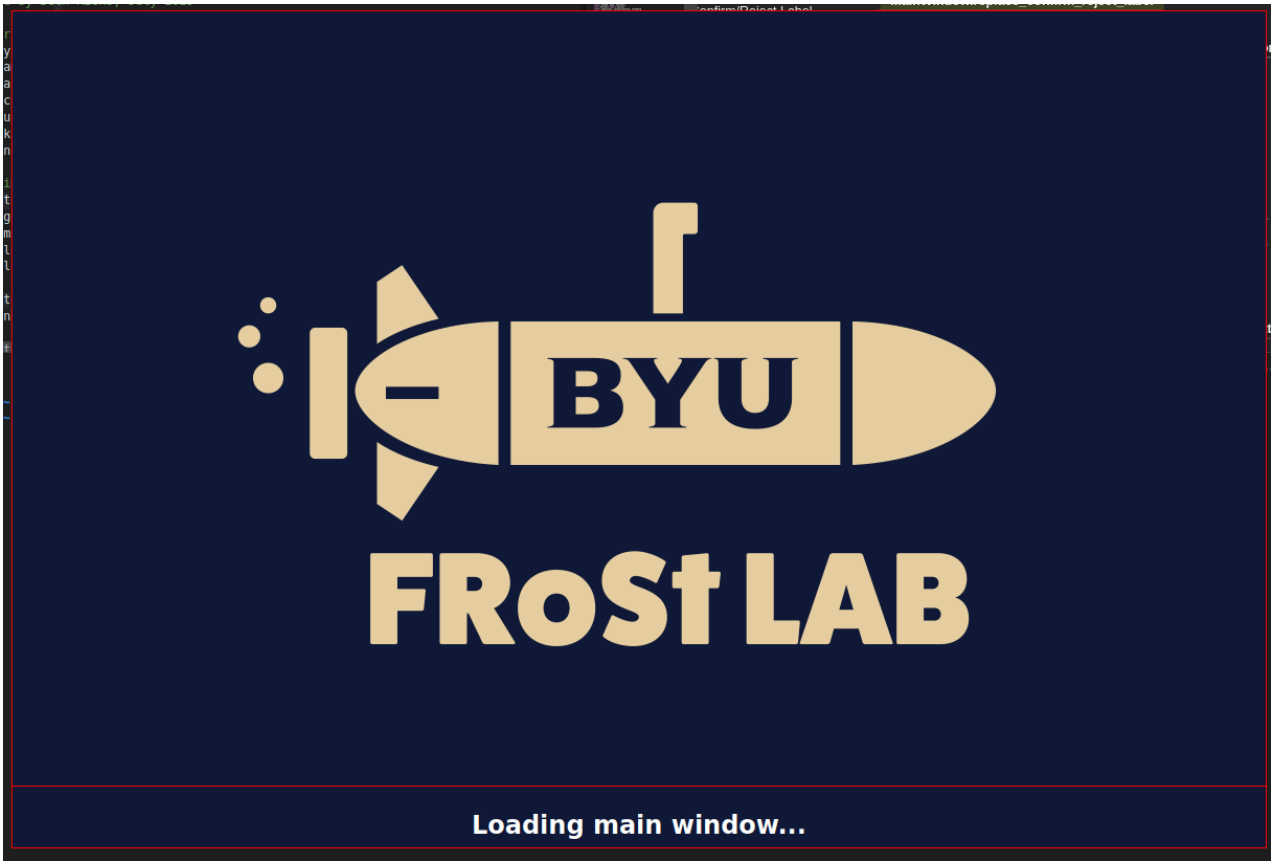
H:

```
def create_specific_vehicle_console_log(self, vehicle_number):
    # Whole method is relevant
```

 GUI General Page Layout >

Splash Screen Layout

Seth Ricks



The spash message (Loading main window) is created here:

```
def OpenWindow(ros_node, borders=False):
    ...
    splash.showMessage("Loading main window...")
    ...
```

For the rest of the splash image (the color, size, splash raising) see the rest of the "OpenWindow" function.

For the splash image, see this page:

❄ Frost Lab png ›

GUI Signals Mind Map

Seth Ricks

Updated 7/16/2025

This Mind Map uses MarkMap. See here for more details: MarkMap

4KB

GUI2_mind_map.zip
archive

This zip folder includes a mark down file used to create the map, and an html of the map itslef.

This map represents all of the signals that the GUI is receiving to update its information, as well as the signals it is sending out to the vehicles/base station.

This map can also be found in:

```
~/cougars/cougars-base-station/base-station-
ros2/src/base_station_gui2/base_station_gui2/GUI_mindmap.mm.md
```

NOTE: This mind map may not be comprehensive. I've done the best I can to document all the signals, but it is possible I have missed some.

GUI Code

Seth Ricks

This page is meant as a rundown for code in the GUI. The following subpages are included:

🚂 Starting up the GUI >

A guide to how to get the GUI up and running

📁 Files and Folders >

A reference for the files and folders that are used in the GUI. An explanation of their purpose and usage.

⚠ Typical Errors >

A couple of errors that are seen often when trying to run the GUI, and how to solve them.

Starting up the GUI

Seth Ricks

First, make sure that you are in the right docker container. It is called 'cougars_base.'

NOTE: The GUI does not currently work with Docker Desktop. You must use Docker Engine.

Then, go to the main branch in cougars-base-station.

If you are on windows, you will have to use wsl. Follow instructions on how to do that here:

Before entering the Docker Container, run the following command:

```
xhost +local:docker
```

This gives docker permission to use the display.

Enter the docker container with:

```
docker exec -it cougars_base bash
```

Then, enter the following folder:

```
cd base_station/base-station-ros2/
```

Now build the packages

```
bash colcon_build.sh
```

Remember to set up the environment with:

```
source install/setup.sh
```

You should now be ready to run the GUI. This can be done by running this command if you want to run it standalone:

```
ros2 run base_station_gui2 talker
```

The GUI should pop up! You can also run the GUI inside of the terminal launch file like this, if you want to run it with all of the other nodes:

```
ros2 launch launch/terminal_launch.py
```

Hopefully it works for you!

Files and Folders

Seth Ricks

There are a lot of files that are related to the GUI that I have worked on this summer. I've tried to break it down into sections for simplicity.

ROS Related:

- 🤖 ROS Files

Folders:

- 🐈 Cougars Bringup
- 🎮 Temp Mission Control
- 📉 Temp Waypoint Planner
- 👂 Vehicles Calibrate

Files:

- ❄️ Frost Lab png
- 👑 GUI ros node
- 🪟 Tabbed Window

I also recommend looking at the two mind maps I created. You can find those here:

🧠 GUI Features Mind Map >

🧠 GUI Signals Mind Map >

ROS Files

Seth Ricks

__init__.py

This is an empty file that makes the folders that it is in a package. It is in most folders in the GUI.

setup.py & package.xml

These are also in some of the packages, they are needed for ROS. For more information about this, see this website: https://docs.ros.org/en/humble/index.html ↗

Cougars Bringup

Seth Ricks

The purpose of this folder is to execute the "Start Missions" command. The file "startup_call.py" is an adaptation of a file that Braden wrote for the GUI.

This file uses ROS publisher to tell the vehicles the options that you selected for starting a mission on this line:

```
publisher.publish(msg)
```

And also prints to the console log on this line:

```
ros_node.publish_console_log(f"Start: {msg.start.data}, Rosbag Flag:
{msg.rosbag_flag.data}, Prefix: {msg.rosbag_prefix}, Thruster:
{msg.thruster_arm.data}, DVL: {msg.dvl_acoustics.data}", vehicle)
```

Both of these (the "publisher" node and the "publish_console_log" function) come from the passed ros_node. This node is created in GUI_ros_node.py, passed into tabbed_window.py, and then passed into startup_call.py

Regardless of whether you selected all vehicles to start their missions or for one specific vehicle to start its mission (in its specific tab page), this file is used. If it is one vehicle [example_number] is passed through sel_vehicles, and if it is all of them [selected vehicle list] is passed through.

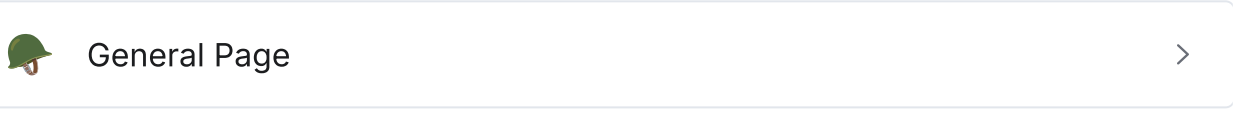
Temp Mission Control

Seth Ricks

The purpose of the temp_mission_control package is to aid in the function of "loading the missions." This is done in the deploy.py script, which is an adaptation of a file in the new_init branch that Braden wrote.

This file uses deploy_config.json, which is some info such as IP addresses, vehicle names, etc;

To see info about the functionality of loading the missions, see the "General Page" Documentation.



In short, the load missions uses deploy.py to load the mission file, the vehicle parameters, and the fleet parameters into the vehicles. If you have issues with having to put passwords into the terminal repeatedly in order to use this funciton, see the "Typical Errors" Documentation.

missions folder:

The missions subfolder in temp_mission_control is where some of the manual json mission files are saved. Although this folder doesn't have to be used when choosing a mission file, I recommend it for simplicity.

params folder:

The params folder is where the parameters for the vehicles is stored. vehicle_params.yaml is the template for the vehicles. If there isn't a parameters file already for the vehicles, you can click on "calibrate fins" to create one for all the vehicles, based off of the vehicle_params.yaml file. This is a backwards way to do it, and making this a separate process is on the GUI TODO list.

🔨 GUI TODO >

Temp Waypoint Planner

Seth Ricks

The purpose of the Temp Waypoint Planner is to open the file (named temp_waypoint_planner.py) that Brighton created. On this planner you can plot an origin and waypoints, and save it to a yaml file.

For more information about how this works, see the Waypoint Missions Documentation here:

Waypoint Missions >

Vehicles Calibrate

Seth Ricks

The purpose of the vehicles_calibrate package is to run a python and base station GUI adapted version of calibrate.sh script found here:

```
~/cougars/cougars-ros2/calibrate.sh
```

This is what I was working on when I finished my internship. It is still pretty buggy, meaning that I haven't gotten it to work with the vehicles yet. The calibrate.py runs with no errors, though. Fixing this and making it work is on the list of GUI TODO.

🔨 GUI TODO >

Frost Lab png

Seth Ricks

The FRoSt LAB png that is used in the splash screen is included below. If this is no longer wanted, simply delete it in the file system and a blank screen will pop up instead. If you want to replace it, the reference for it can be found in tabbed_window.py in the "OpenWindow" Function, on this line:

```
def OpenWindow(ros_node, borders=False):
    ...
    # Prepare splash image
    img_path = os.path.expanduser("~/base_station/base-station-
ros2/src/base_station_gui2/base_station_gui2/FRoSt_Lab.png")
    ...
```

GUI ros node

Seth Ricks

The file GUI_ros_node is where the GUI is launched from. If you look in the setup.py, you'll see this line:

```
entry_points={
    'console_scripts': [
            'talker = base_station_gui2.GUI_ros_node:main'
    ],
},
```

This means that the 'main' function in GUI_ros_node is the entry point for ros, and it is labeled as 'talker.' This is why, after building and sourcing, you can launch the GUI using this command:

```
ros2 run base_station_gui2 talker
```

GUI_ros_node holds the ROS node that connects all of the GUI together. This means that all of the subscribers, publishers, services, and other methods pertaining to the node are in this file. This is why the GUI node is passed around to so many places in packages such as temp_mission_control, so that they can access the messages in the node. I have found that starting a ROS node from another node causes a lot of issues, hence why there is one file that contains everything the GUI needs.

I did the best I can to add good comments throughout this file to explain what is going on, along with function descriptions. Just remember that all this file is trying to do is create a ros node, configure it, start the GUI, and pass in the ros node to the GUI.

Note: The function SeeAllIcons() is purely for development, and can be removed if desired. All it does is pop up a window showing all of the built-in icons that PyQt6 has to offer. I found it useful because these icons can change depending on your os, but it is up to you if you want to use it or not.

Tabbed Window

Seth Ricks

This file is daunting, even for me. It's currently over 3000 lines long, so I will try to break it up into chunks as I explain it. I highly recommend that you go and study how PyQt6 works first before trying to understand what this file is doing. The PyQt6 documentation that I used can be found here: https://www.pythonguis.com/tutorials/pyqt6-first-steps-qt-designer/↗

The main purpose of this file is to do all the nitty gritty configurations in the GUI. The main brunt of it is purely for visual configuration. This includes the colors, shapes, and sizes off all of the widgets and layouts.

The second biggest (and most important) thing that this file does is connect all of the *signals* of the GUI together. This means that if you press the 'Load All Missions' button, it triggers a signal that executes the logic related to loading all of the missions. Almost nobody likes an app that looks ugly, but **everybody** hates an app that doesn't work right.

I have broken up the explanation of this file into three pages, based off these two purposes. You can get to those pages here:

🧠 GUI Features Mind Map >

🐕‍🦺 GUI Layout Guide >

🧠 GUI Signals Mind Map >

Typical Errors

Seth Ricks

xcb-cursor0 error:

This is a very typical error found when trying to run the GUI inside of docker. Docker doesn't initially have permissions to the display.

```
Authorization required, but no authorization protocol specified
qt.qpa.xcb: could not connect to display :0
qt.qpa.plugin: From 6.5.0, xcb-cursor0 or libxcb-cursor0 is needed to
load the Qt xcb platform plugin.
qt.qpa.plugin: Could not load the Qt platform plugin "xcb" in "" even
though it was found.
This application failed to start because no Qt platform plugin could be
initialized. Reinstalling the application may fix this problem.

Available platform plugins are: eglfs, wayland-egl, vnc, wayland,
minimal, offscreen, xcb, minimalegl, vkkhrdisplay, linuxfb.
```

To fix this, exit Docker and use the command:

```
xhost +local:docker
```

or

```
xhost +
```

This gives docker permission to use the display. This must be done every time you reboot your computer, restart the X server, switch users and/or sessions, or you log out and log back in to your desktop environment.

Module not found error:

This error is similar to:

```
ModuleNotFoundError: No module named 'pyqt6'
```

This most likely means that you are in the wrong docker image. The image is cougars_base. If you need to stay in the image you are already in (NOT recommended), you can install the dependencies for the GUI in the script located here:

```
~/cougars/cougars-base-station/base-station-ros2/gui_docker_setup.sh
```

Just run this script inside of docker:

```
bash gui_docker_setup.sh
```

Password requested when ssh-ing:

One of the known issues with accessing the vehicles is the automation of ssh-ing. Operating inside the vehicle directory can take multiple 'frostlab' passwords. So if something like 'loading the missions' or 'copy bags' seems "stuck" it's probably waiting for a password in the terminal. You can put the password in a bunch, or you can volume the vehicle into the docker container, which I don't totally know how to do. What I have usually done for a temporary fix is copy the vehicles ssh key to the docker container for that session. To do this, stop the processes you are running, and inside the Docker container run this command:

```
ssh-keygen (press enter for all the prompts)
ssh-copy-id frostlab@192.168.0.102 (or whatever the vehicle IP address
is)
```

GUI Features

Seth Ricks

This page is meant to be a run down of all of the features that the GUI currently has. The following subpages are included:

📎 Configuration Window >

An explanation of the usage of the configuration window that first pops up when you launch the GUI.

🎨 Theme Button >

This explains how to change the colors of the GUI.

🪖 General Page >

An explanation of the features on the general page of the GUI.

🧐 Specific Page(s) >

An explanation of the features on the specific page(s) of the GUI.

🧠 GUI Features Mind Map >

A guide to how to locate features of the GUI in the code for editing.

NOTE: Visuals of all of these features can be seen here:

👀 How it looks >

Configuration Window

Seth Ricks

The configuration window is pretty simple. This window is on top of the splash screen, and prompts you to choose the vehicles that you would like to include in the program. Only integers are accepted, in the range of 0-999.

Vehicle 3 is currently designed to be the BlueROV, because of the IP address in deploy_config.json found at this path:

```
~/cougars/cougars-base-station/base-station-ros2/src/base_station_gui2/base_station_gui2/temp_mission_control/deploy_config.json
```

Currently a maximum of 4 vehicles can be selected. This is because the GUI will become too "wide" for the screen if more than 4 are chosen. Being able to chose more than 4 is on the GUI TODO list.

Vehicle 0 is meant run the GUI with Holoocean, but it is currently in development. This is on the GUI TODO list.

🔨 GUI TODO >

Theme Button

Seth Ricks

The "Theme" button is located in the top left corner of the GUI. Clicking on it will bring up a submenu called "Set Theme." There are 7 color themes: Dark Mode, Light Mode, Blue Pastel, Brown Sepia, Intense Dark, Intense Light, and Cadetblue. These themes are designed for usage in indoor/outdoor environments. All of these themes are ADA approved, exempting Cadetblue.

Bonus: Once you get past the splash page, try typing "duckietown" 😉

General Page

Seth Ricks

Most of the information that is on the General Page is also on the Specific Pages. The General Page is designed to oversee all of the vehicles in the mission at a time, without having to change tabs. There are a few unique features on this page that are explained here.

Load All Missions:

Clicking on this button brings up a pop-up window. On this window you can choose a vehicle with the tabs, and then browse for files. This will bring up another window with your file system. Choose a file to load, and press open. Optionally, you can choose to apply the mission file you selected to all of the vehicles.

When you press 'okay', a few things will happen. First, the GUI will load the mission files you selected to the vehicles over Wifi. It does this using ssh. It first deletes the old mission file on the vehicle, and then copies the new one over. It does this same process for the params files, and for the vehicle params files.

In the console log of the vehicle, you will be able to see if loading the missions was a success or not. I have found that if the wifi connections for the base station GUI and the vehicle are both good and on the same network, this loading works very well.

The "Load All Missions" Button loads the missions for all of the vehicles at the same time. If you are wanting to load only one mission into one vehicle, then select the button "Load Mission" on that vehicle's specific page.

After clicking 'okay' on the Load All Missions button, map_viz_path and map_viz_origin data are also published. This is for the separate map viz window that is in development for plotting the waypoint missions. The map viz path and origin are NOT published if you do a "Load Mission" button on one of the vehicle-specific pages. Testing if this works is still on the GUI TODO list.

Note: If this command seems "stuck," meaning it isn't getting past the first part of the process, it is probably waiting for a password in the terminal to do something inside of the vehicle's directory. See the "Typical Errors" page.

Start All Missions:

Clicking on start all missions button brings up a pop-up window with configuration options. There are 5 different options: Start the node, Record rosbag, Rosbag prefix, Arm Thruster, and Start DVL. Any of these are optional. The only restraint is that if you select record rosbag, you must enter a prefix, and visa versa.

The "Start All Missions" Button starts the missions for all of the vehicles at the same time. If you are wanting to start only one mission on one specific vehicle, then select the button "Start Mission" on that vehicle's specific page.

Plot Waypoints:

This button is unique to the General Page. When this button is clicked, a pop-up window appears with a map on the right side. This window is meant to plot a waypoint mission. For use of this window, see this page:

Waypoint Missions >

When you have completed planning a mission, save the file to a desired directory. Remeber this directory, because if you want to load this mission from the GUI later you will need to know where it is.

Copy Bags to Base Station:

Clicking on this button will copy the bag folder on the vehicles to the base station. As long as there is a connections, this works very well.

If you want to copy bags from a specific vehicle and not all of them at once, select the button "Copy Bag to Base Station" on that vehicle's specific page.

Note: If this command seems "stuck," meaning it isn't getting past the first part of the process, it is probably waiting for a password in the terminal to do something

inside of the vehicle's directory. See the "Typical Errors" page.

Calibrate All Vehicles:

This process is very buggy and still in development. It is on the list of GUI TODO.

What this button is *supposed* to do is run some bash scripts to connect to the vehicles and calibrate depth, the DVL gyro, and set the ntp. But it doesnt seem to be working yet.

Calibrate Fins:

Calibrating the Fins is also in progress, and on the list of GUI TODO. Clicking on this will bring up the fin calibration window. On this window are tabs for the different vehicles, and three sliders for the three different fin positions. The process tries to connect to the vehicle and get the current fin positions, if it can't do that then it will try to get it from the base station, and if that doesn't exist it will make a new parameters file, copying vehicle_params inside of temp_mission_control → params and creating a new file called "cougX_params.yaml" in the same directory. This all seems to work, the fins will even move real-time as you change the sliders! You can also use the up/down/pg up/ pg down buttons on your keyboard, as the window says.

What needs to be done has to do with the buttons on the bottom. You are supposed to be able to change the publisher for the ros2 fin data, but it isn't working as desired. When the window closes it is also supposed to ros2 param set the fins on the vehicles, but that isn't working either.

Note: If this command seems "stuck," meaning it isn't getting past the first part of the process, it is probably waiting for a password in the terminal to do something inside of the vehicle's directory. See the "Typical Errors" page.

Recall Vehicles (No Signal):

This button doesn't do anything yet. It is supposed to recall the vehicles back to the origin via the modem. There is a similar button on the specific pages. Both of these are on the GUI TODO list.

Confirm/Reject Label:

The Confirm/Reject label is on the bottom left of the GUI. This label is helpful to know commands that are going on throughout the GUI without having to change pages. It is the same throughout the window, meaning that it will say the same thing on every page. It is constantly being updated when something has happened, so most of the time it will represent the last thing that finished on the GUI.

Icons:

All of the icons are the same for the vehicles' information on both the General and the Specific Pages. Meaning, if there is a checkmark next to Vehicle 1's Wifi label, there will also be a checkmark next to the wifi icon on the vehicles' specific page. This goes for all of the icons. A checkmark means that there is a connection, a red 'x' means that there is not a connection, and a question mark means that no data has been recieved yet.

Connections:

> Wifi Icon:
>
> The Wifi Icon gets its information by trying to ping the vehicles every 3 seconds. If the connection status has changed, this information is published to the modem. This is done so that the modem can know whether or not it needs to be on or not. This icon will need to be a checkmark in order to do many of the functions of the GUI.

> Radio/Modem Icons: The icon for this message are from the "connections" message. You can find the location of this message type in the GUI Mind Map. These work well.

Sensors:

The status for the following sensors come from the SystemStatus message:

> DVL
>
> GPS
>
> IMU

These seem to work well. The SystemStatus message is inside of frost_interfaces.

Emergency Status:

The Emergency Status label is supposed to tell you if there is a major problem with one of the vehicles or not. This also comes from the SystemStatus message.

🧠 GUI Signals Mind Map >

🔨 GUI TODO >

Typical Errors >

Specific Page(s)

Seth Ricks

Many of the features on the Specific Pages can be found on the General Page. There are a few features that are unique to the Specific Page, which will be explained here.

For the following features, see the General Page description:

> Connections Icons
>
> Sensor Icons
>
> Load Mission
>
> Start Mission
>
> Copy Bag to Base Station
>
> Calibrate Vehicle
>
> Recall Vehicle
>
> Confirm/Reject Label

The following features are unique to the Specific Page and will be explained here:

> Status
>
> Seconds since last connected
>
> Emergency Surface
>
> Emergency Shutdown
>
> Console information/message log
>
> Clear Console

Status:

To see where the following status messages come from, or if they have a signal yet, see the GUI Mindmap.

> x
>
> y
>
> Depth
>
> Heading
>
> Current Waypoint (NOTE: Has no signal yet.)
>
> DVL Velocity
>
> Angular Velocity
>
> Battery
>
> Pressure

Almost all of these haven't been tested yet. But Pressure and Depth are very reliable. Making sure that the rest of them work and are recieving the correct message types is on the GUI TODO list.

Seconds since last connected:

The 'seconds since last connected' labels for the radio and the accoustics is from the Connections message inside of base station interfaces. It seems to be working well, but the correct data isn't being published yet.

Emergency Surface:

This service is supposed to surface the vehicle via the base station modem, but it hasn't been tested in the field yet. It seems to work in the lab and in the tank. This data is from the BeaconId service inside of base station interfaces.

Emergency Shutdown:

This service is supposed to 'kill' the vehicle, but it hasn't been tested yet in the field. This data is from the BeaconId service inside of base station interfaces.

Console information/message log:

The console log is for messages pertaining to that vehicle, or all of the vehicles. It can be used to track if the commands you are sending are actually working. You can write to this console from another running program using the ConsoleLog message, which can be found in base station interfaces. I recommend looking at the Mind Map for more clarity.

Clear Console:

The clear console button is self-explanatory. It simply clears the console for that vehicle if needed. Just know that this can't be undone.

🧠 GUI Signals Mind Map >

🪖 General Page >

GUI Features Mind Map

Seth Ricks

- Go through mind map to make sure it is all accurate

The following zip file contains another mind map, that is specifically about the features of the GUI that was explained in the other sections of "GUI Features." It can be helpful for you to know where the implementation of certain aspects of the GUI can be related in the code. All of this is talking solely about tabbed_window.py.

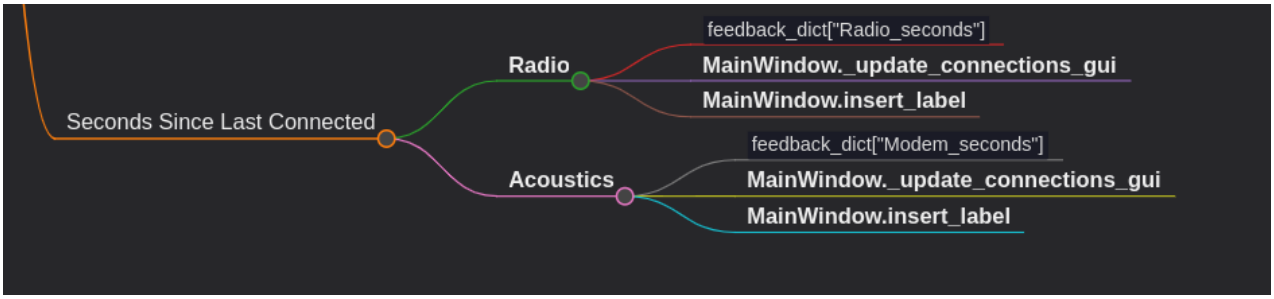
5KB

GUI_features_mindmap.zip
archive

It can also be located in the following directory:

```
~/cougars/cougars-base-station/base-station-
ros2/src/base_station_gui2/base_station_gui2/tabbed_window.mm.md
```

For example, if you wanted to work on the "Seconds Since Last Connected" label, then you can look at the branch below. Notice how "Radio" has three branches, each pointing to aspects that pertain to this specific label. If you wanted to edit this label, then you would want to go one of three places: the feedback_dict, _ update_connections_gui, or insert_label.



Note: If you are looking more into changing the *layout* of the GUI, as in where to find where all the features are created, there is a separate documentation for that. It would be helpful to go there if you want to add/remove widgets or change the layout of the GUI. You can find the guide to that here:

GUI Layout Guide

GUI TODO

Seth Ricks

- [ ] Add a button for opening MapViz (w/ Eli)
- [ ] Test that vehicle 0 works with Holoocean (w/ Brighton)
- [ ] More than 4 vehicles can be selected
- [ ] Fix the bugs in and test Calibrate All Vehicles (w/ Braden)
- [ ] Fix the issues with Calibrate Fins
- [ ] Create logic with Recall Vehicles
- [ ] Test Emergency surface/kill (w/ Ben W.)
- [ ] Create Logic for receiving the "Last Waypoint" label under status
- [ ] Test all the Status Messages, make sure they are the correct messages being received
- [ ] Create "create params" button, copying from vehicle_params.yaml (currently using "calibrate fins" to create new params)